LAPORAN PRAKTIKUM ANALISIS KOMPLEKSITAS ALGORITMA

Oleh :

MUHAMMAD RAJUL (2025573010123)

KELAS : 1D

PRODI : TEKNIK INFORMATIKA

JURUSAN : TEKNOLOGY INFORMASI KOMPUTER

DOSEN : M REZA ZULMAN S.ST, M.SC

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFOMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE

TAHUN AJARAN 2026/2027

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum dengan judul *"Analisis Kompleksitas Algoritma"* ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu tugas pada mata kuliah Struktur Data dan Algoritma. Tujuan dari penyusunan laporan ini adalah untuk memahami konsep Big-O Notation serta menganalisis kinerja algoritma berdasarkan waktu eksekusi terhadap berbagai ukuran data.

Dalam proses penyusunan laporan ini, saya menyadari masih terdapat kekurangan. Oleh karena itu, saya mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik di masa yang akan datang.

Akhir kata, saya berharap laporan ini dapat memberikan manfaat dan menambah pemahaman mengenai analisis kompleksitas algoritma.

Buketrata, _____ 2026

(Muhammad Rajul)

KATA PENGANTAR .....

DAFTAR ISI .....

BAB I PENDAHULUAN

1.1 Latar Belakang .....

1.2 Rumusan Masalah .....

1.3 Tujuan .....

BAB II DASAR TEORI .....

BAB III METODOLOGI PRAKTIKUM .....

BAB lV PENUTUP .....

- BAB I PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, efisiensi algoritma sangat penting untuk menentukan seberapa cepat suatu program dapat berjalan. Semakin besar data yang diproses, maka semakin besar pula waktu yang dibutuhkan jika algoritma yang digunakan tidak efisien.
Oleh karena itu, diperlukan suatu cara untuk mengukur dan menganalisis performa algoritma, yaitu dengan menggunakan Big-O Notation. Dengan memahami kompleksitas algoritma, kita dapat memilih solusi yang lebih optimal dalam menyelesaikan suatu permasalahan.

1.2 Rumusan Masalah

Adapun rumusan masalah dalam praktikum ini adalah:

1. Apa yang dimaksud dengan kompleksitas algoritma?
2. Bagaimana cara mengukur waktu eksekusi suatu algoritma?
3. Bagaimana perbandingan performa algoritma dengan kompleksitas O(1), O(n), O(n log n), dan $O(n^2)$?

1.3 Tujuan

Tujuan dari praktikum ini adalah:

1. Memahami konsep Big-O Notation
2. Menganalisis waktu eksekusi algoritma
3. Membandingkan beberapa jenis kompleksitas algoritma
4. Mengetahui algoritma yang lebih efisien

➢ BAB II

DASAR TEORI

Big-O Notation adalah metode untuk mengukur efisiensi algoritma berdasarkan waktu eksekusi terhadap ukuran input (n). Notasi ini digunakan untuk mengetahui seberapa cepat atau lambat suatu algoritma bekerja ketika data semakin besar.

Jenis-jenis kompleksitas algoritma:

- **O(1)** (konstan): waktu tetap, tidak dipengaruhi jumlah data
- **O(n)** (linear): waktu bertambah seiring jumlah data
- **O(n log n)** (linear-logaritmik): kombinasi loop dan log
- **$O(n^2)$** (kuadratik): nested loop, sangat lambat untuk data besar

Semakin kecil kompleksitas, semakin efisien algoritma tersebut.

➢ BAB III

METODE PRAKTIKUM

3.1 Alat dan Bahan

Alat:

1. Komputer/Laptop
2. Keyboard & Mouse
3. Akses Internet

Bahan:

1. Git Bash
2. Visual Studio Code

3.2 Langkah-Langkah Praktikum

1. Membuka Git Bash sebagai terminal kerja.
2. Berpindah ke direktori tempat repository berada dengan perintah ***cd*** misalnya masuk ke D lakukan seperti gambar tersebut

```
LOQ@LAPTOP-Q2E352VR MINGW64 ~
$ cd /d/
```

3. Masuk ke folder repository yang sudah di-clone,

```
LOQ@LAPTOP-Q2E352VR MINGW64 /d
$ cd pratikumalgoritma
```

4. Tuliskan nama repository kamu seperti di gambar

```
OQ@LAPTOP-Q2E352VR MINGW64 /d/pratikumalgoritma
$ cd 2025573010123_strukturdataalgoritma
```

5. Membuat folder baru untuk praktikum dengan ***mkdir.***

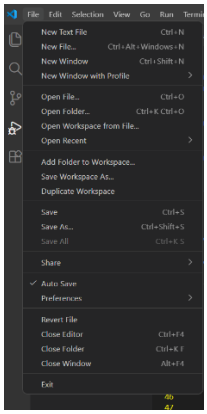
```
LOQ@LAPTOP-Q2E352VR MINGW64 /d/pratikumalgoritma/2025573010123_strukturdataalgor
itma (main)
$ mkdir pratikum-5
```

6. Menginisialisasi NodeJS dengan perintah ***npm init -y***

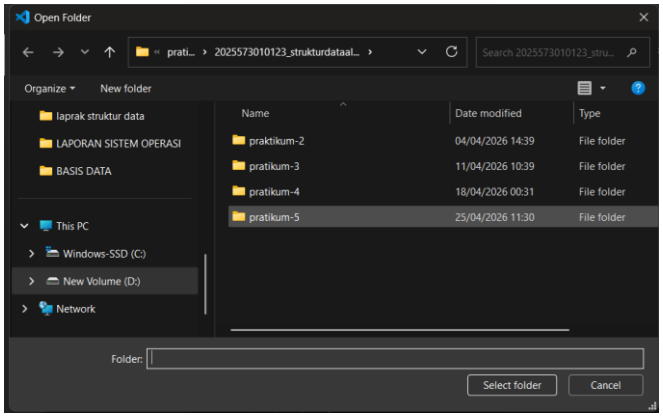
```
LOQ@LAPTOP-Q2E352VR MINGW64 /d/pratikumalgoritma/2025573010123_strukturdataalgor
itma/pratikum-5 (main)
$ npm init -y
Wrote to D:\pratikumalgoritma\2025573010123_strukturdataalgoritma\pratikum-5\pac
kage.json:

{
  "name": "pratikum-5",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "isc",
  "type": "commonjs"
}
```

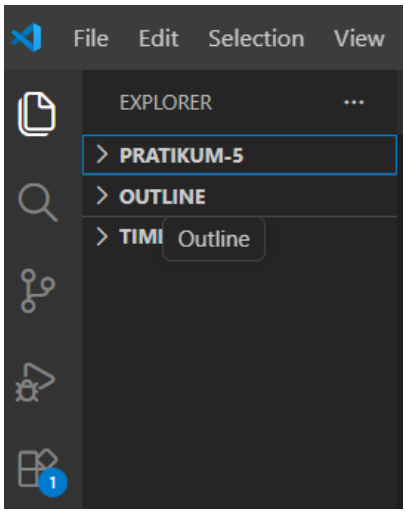
7. Buka folder yang udah di tambah di git di vs code
    a. Buka vs code arahkan cursor ke file cari open folder



    b. Dan cari folder yang di buat tadi di gitbash



c. Hasilnya bakalan seperti ini

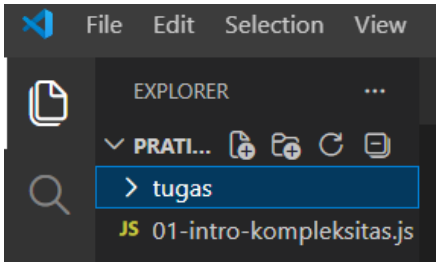


d. Kembali ke gitbash buat file di dalam folder tersebut menggunakan ***touch***

Seperti di gambar

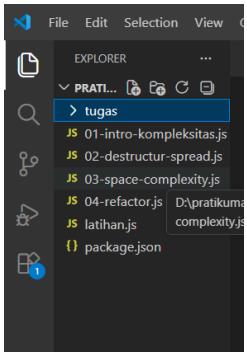
```
LOQ@LAPTOP-Q2E352VR MINGW64 /d/pratikumalgoritma/2025573010123_strukturdataalgor
itma/pratikum-5 (main)
$ touch 01-intro-kompleksitas.js
```

e. Cek di vs code otomatis bakalan adaa file dalam folder pratikum-5



Dan begini hasilnyaa

f. Masukin semua file yg di suruh dan hasilnya seperti ini :

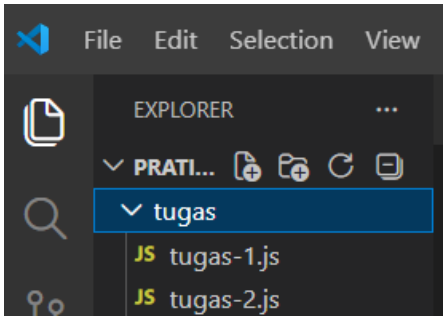


g. Ketik program sesuai file yg dinamakan
h. Selanjutnya bikin nama folder baru di repositpry yang sama dengan nama tugas dan di dalam nyaa ada file tugas 1 dan 2
Seperti

```
LOQ@LAPTOP-Q2E352VR MINGW64 /d/pratikumalgoritma/2025573010123_strukturdataalgor
itma/pratikum-5 (main)
$ mkdir tugas

LOQ@LAPTOP-Q2E352VR MINGW64 /d/pratikumalgoritma/2025573010123_strukturdataalgor
itma/pratikum-5 (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js
```

Hasilnya di vs code



INPUT TUGAS SATU

```
function interLambat(a, b){
  let hasil = [];

  for(let i=0; i<a.length; i++){
    for(let j=0; j<b.length; j++){
      if(a[i] == b[j]){
        hasil.push(a[i]);
      }
    }
  }

  return hasil;
}

function interCepat(a, b){
  let hasil = [];
```

```
  let set = new Set(a);

  for(let i=0; i<b.length; i++){
    if(set.has(b[i])){
      hasil.push(b[i]);
    }
  }

  return hasil;
}



function anagram(arr){
  let obj = {};

  for(let i=0; i<arr.length; i++){
    let kata = arr[i];
    let key = kata.split('').sort().join('');

    if(obj[key]){
      obj[key].push(kata);
    } else {
      obj[key] = [kata];
    }
  }

  return Object.values(obj);
}

function tripletLambat(arr){
  for(let i=0;i<arr.length;i++){
    for(let j=0;j<arr.length;j++){
      for(let k=0;k<arr.length;k++){
        if(arr[i]*arr[i] + arr[j]*arr[j] == arr[k]*arr[k]){
          return true;
        }
      }
    }
  }
  return false;
}
```

```
function tripletCepat(arr){
  arr.sort((a,b)=>a-b);

  for(let i=arr.length-1;i>=0;i--){
    let kiri = 0;
    let kanan = i-1;

    while(kiri < kanan){
      let jumlah = arr[kiri]*arr[kiri] + arr[kanan]*arr[kanan];
      let target = arr[i]*arr[i];

      if(jumlah == target){
        return true;
      } else if(jumlah < target){
        kiri++;
      } else {
        kanan--;
      }
    }
  }

  return false;
}

    console.log ("=== hasil ===");
let a = [1,2,3,4];
let b = [3,4,5];

console.log("Inter lambat:", interLambat(a,b));
console.log("Inter cepat:", interCepat(a,b));

let kata = ["eat","tea","tan","ate","nat","bat"];
console.log("Anagram:", anagram(kata));

let angka = [3,4,5];
console.log("Triplet lambat:", tripletLambat(angka));
console.log("Triplet cepat:", tripletCepat(angka));
```

OUTPUT

```
=== hasil ===
Inter lambat: [ 3, 4 ]
Inter cepat: [ 3, 4 ]
Anagram: [ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]
Triplet lambat: true
Triplet cepat: true
```

INPUT TUGAS 2

```
function f1(n){
  return n + 1;
}

function f2(n){
  let total = 0;
  for(let i=0; i<n; i++){
    total += i;
  }
  return total;
}

function f3(n){
  let count = 0;

  for(let i=0; i<n; i++){
    for(let j=1; j<n; j*=2){
      count++;
    }
  }

  return count;
}

function f4(n){
  let count = 0;

  for(let i=0; i<n; i++){
    for(let j=0; j<n; j++){
      count++;
    }
  }

  return count;
}

function ukur(fn, n){
  let mulai = Date.now();
  fn(n);
  let selesai = Date.now();
  return selesai - mulai;
```

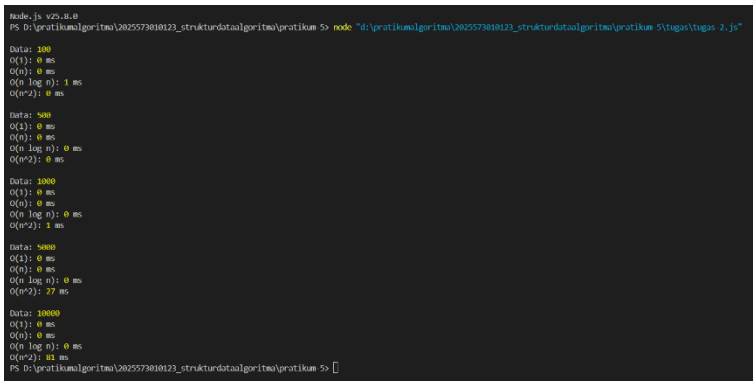
```
}

function benchmarkSemua(ukuranData){
  for(let i=0; i<ukuranData.length; i++){
    let n = ukuranData[i];

    console.log("\nData:", n);
    console.log("O(1):", ukur(f1, n), "ms");
    console.log("O(n):", ukur(f2, n), "ms");
    console.log("O(n log n):", ukur(f3, n), "ms");
    console.log("O(n^2):", ukur(f4, n), "ms");
  }
}

benchmarkSemua([100, 500, 1000, 5000, 10000]);
```

OUTPUT



```
Node.js v25.8.0
PS D:\pratikumalgoritma\202557301012 3_strukturdataalgoritma\pratikum 5> node "d:\pratikumalgoritma\202557301012 3_strukturdataalgoritma\pratikum 5\tugas\tugas 2.js"

Data: 100
O(1): 0 ms
O(n): 0 ms
O(n log n): 1 ms
O(n^2): 0 ms

Data: 500
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 0 ms

Data: 1000
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 1 ms

Data: 5000
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 27 ms

Data: 10000
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 81 ms
PS D:\pratikumalgoritma\202557301012 3_strukturdataalgoritma\pratikum 5>
```

8. setelah semua selesai summit ke git hub

   Caranyaa

   1. Buka git bash
   2. Masuk ke c atau dimna folder tadi disimpen
   3. Masuk ke repository
   4. Ketik cd git status
   5. Dan git add pratikum-5/
   6. Ketik : git commit -m “ pratikum 5 : Analisis Kompleksitas Algoritma"
   7. Dan terakhir ketik git push origin main
   8. Tunggu sampe semua selesai dan hasilnya seperti ini

```
LOQ@LAPTOP-Q2E352VR MINGW64 /d/pratikumalgoritma/2025573010123_strukturdataalgor
itma (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   pratikum-3/tugas/tugas-1.js
        modified:   pratikum-4/03-class-dasar.js

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        pratikum-3/tugas/tempCodeRunnerFile.js
        pratikum-5/

no changes added to commit (use "git add" and/or "git commit -a")

LOQ@LAPTOP-Q2E352VR MINGW64 /d/pratikumalgoritma/2025573010123_strukturdataalgor
itma (main)
$ git add pratikum-5/
warning: in the working copy of 'pratikum-5/package.json', LF will be replaced b
y CRLF the next time Git touches it

LOQ@LAPTOP-Q2E352VR MINGW64 /d/pratikumalgoritma/2025573010123_strukturdataalgor
itma (main)
$ git commit -m "Pratikum 5 : Analisis Kompleksitas Algoritma"
[main 19b9c97] Pratikum 5 : Analisis Kompleksitas Algoritma
 9 files changed, 399 insertions(+)
 create mode 100644 pratikum-5/01-intro-kompleksitas.js
 create mode 100644 pratikum-5/02-destructur-spread.js
 create mode 100644 pratikum-5/03-space-complexity.js
 create mode 100644 pratikum-5/04-refactor.js
 create mode 100644 pratikum-5/latihan.js
 create mode 100644 pratikum-5/package.json
 create mode 100644 pratikum-5/tugas/tempCodeRunnerFile.js
 create mode 100644 pratikum-5/tugas/tugas-1.js
 create mode 100644 pratikum-5/tugas/tugas-2.js

LOQ@LAPTOP-Q2E352VR MINGW64 /d/pratikumalgoritma/2025573010123_strukturdataalgor
itma (main)
$ git push origin main
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 12 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (13/13), 4.37 KiB | 1.46 MiB/s, done.
Total 13 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/muhammadrajul21/2025573010123_strukturdataalgoritma.git
   55c28d0..19b9c97  main -> main
```

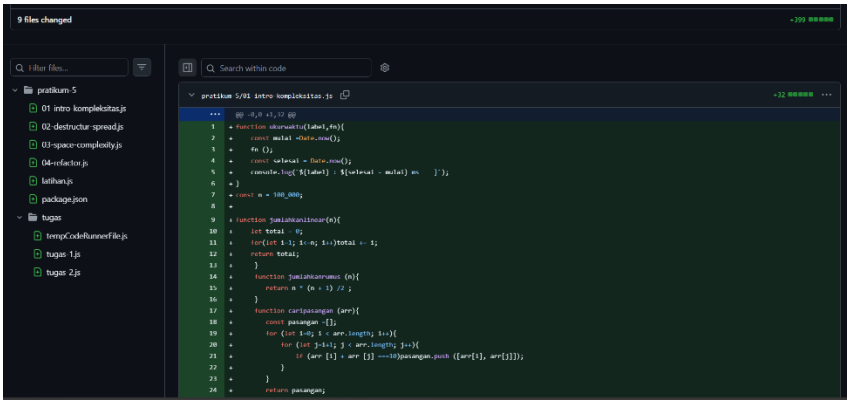
9. Cek di github dan halaman nya bakalan seperti ini di repository kamu

Pratikum 5 sudah berhasil di tambahkan.

10. Cek semua folder yang udah di tambah di pratikum 5

    Cek semua sampe lengkap dan sesuai

    Tampilan nya bakalan seperti ini :



Selesai

➢ BAB IV

PENUTUP

Kesimpulan

1. Kompleksitas algoritma menentukan kecepatan program
2. Algoritma dengan kompleksitas kecil lebih efisien
3. $O(n^2)$ tidak cocok untuk data besar
4. Hasil praktikum sesuai dengan teori Big-O

Saran

1. Gunakan algoritma yang efisien dalam pengolahan data
2. Hindari penggunaan nested loop jika tidak diperlukan
3. Lakukan pengujian performa sebelum memilih algoritma